



Uso del API HTML5 Server-Side Events entre javascript y JakartaEE 10

Autores:

- Maria Lucia Gómez Gutiérrez
- Mario Martínez Gallego
- Pablo Mediano Martínez

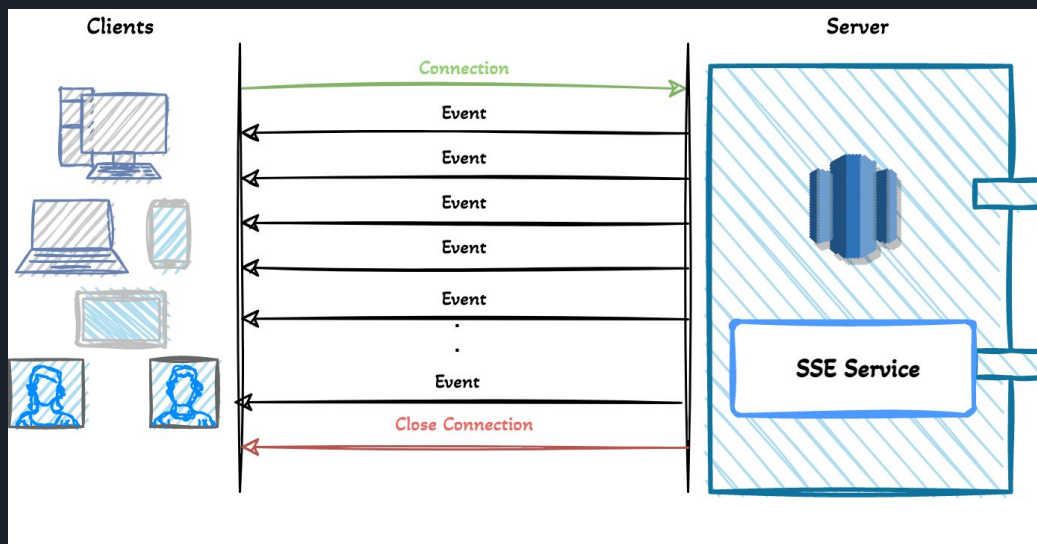
Curso: 3º Ingeniería Informática

Índice

1. [Introducción a los Server-Side Events \(SSE\)](#)
2. [Problemática que resuelve](#)
3. [Arquitectura general](#)
4. [Implementación \(Servidor\)](#)
5. [Cliente JavaScript](#)
6. [Ejemplo funcional](#)
7. [Gestión de errores](#)
8. [Reconexión automática](#)
9. [Ejemplo práctico](#)
10. [Ventajas y limitaciones de SSE](#)
11. [Conclusión](#)
12. [Referencias](#)
13. [IAG generativas utilizadas y su aporte](#)

Introducción a los Server-Side Events

Los Server-Sent Events (SSE) son un estándar del HTML5 que permite la comunicación unidireccional desde el servidor hacia el cliente, utilizando una conexión HTTP persistente.



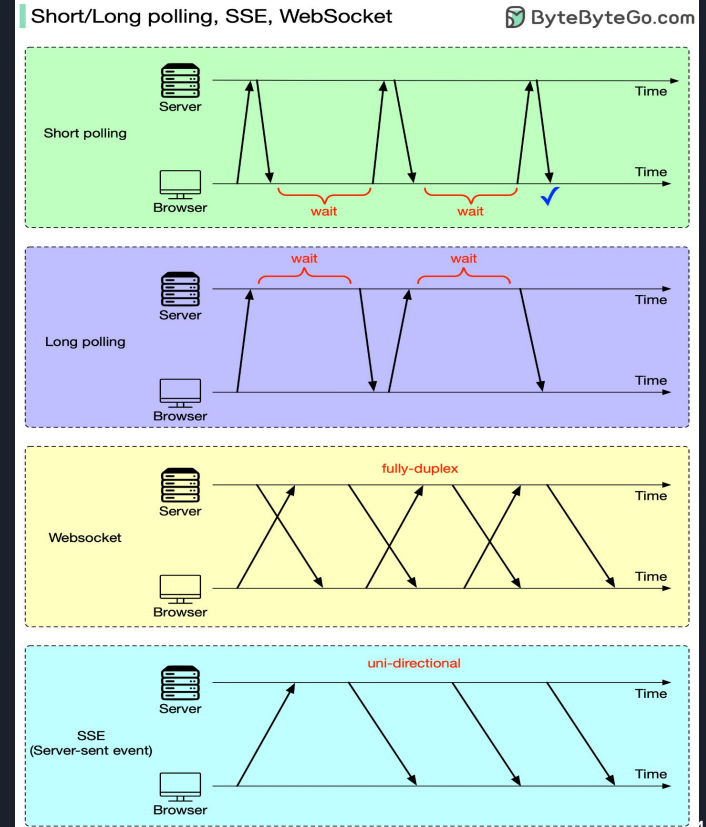
Problemática que resuelve

1 Evolución y eficiencia de la conexión cliente-servidor

SSE hace frente a problemas potenciales a los que están expuestos otros tipos de conexión como el *polling* tradicional y el *long polling*, debido a que estos realizan varias conexiones HTTP según los eventos. Algunos de ellos son:

- Sobrecarga del servidor por peticiones constantes
- Incompatibilidad con navegadores web
- Desperdicio de recursos

También mejora la eficiencia de la web al evitar el uso de *WebSocket*, ya que es una conexión bidireccional, e implica una configuración más compleja y un gasto de recursos innecesarios si buscamos un sistema de eventos que solo recibe el cliente.





Problemática que resuelve

2 Mejoras varias con el uso de HTML5 y Jakarta EE 10 o superior

El uso de *SSE* junto con las versiones más actuales de HTML y Jakarta supera a versiones anteriores por varias razones :

Escalabilidad y modularidad:

Permite gestionar un mayor número de conexiones simultáneas de manera confiable, ideal para aplicaciones en tiempo real.

Facilidad de desarrollo y mantenimiento:

No hay que gestionar manualmente la escritura en streams de salida

Evita el uso de técnicas improvisadas:

Se evita el uso de *iframes*, conexiones abiertas simuladas y otros que daban numerosos problemas y usaban muchos recursos.

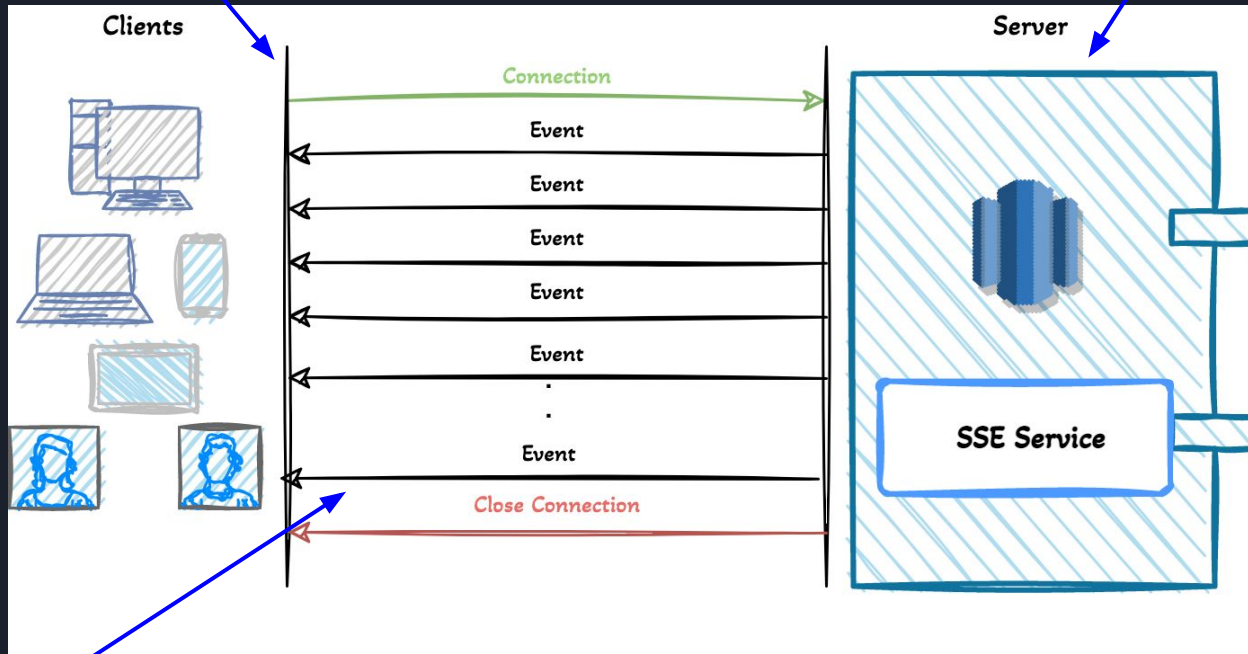
Mejor manejo de conexiones:

La persistencia, estabilidad y eficiencia de la conexión no dependen de soluciones adicionales

Arquitectura general

Se conecta con *EventSource*

Mantiene la conexión HTTP y envía los eventos periódicamente



Maneja los eventos con *onmessage*



Implementación (Servidor)

```

@Path("/events")
@Singleton
public class SseResource {
    @GET
    @Produces(MediaType.SERVER_SENT_EVENTS)
    public void stream(@Context SseEventSink sink, @Context Sse sse) {
        new Thread() -> {
            try (SseEventSink session = sink) {
                // Enviar 10 eventos "data"
                for (int i = 1; i <= 10; i++) {
                    session.send(sse.newEventBuilder()
                        .name(s: "mensaje")
                        .data(String.class, o: "Hola mundo " + i + "!")
                        .build());
                    Thread.sleep(millis: 1000);
                }
                // Mantener la conexión abierta enviando comentarios
                while (!session.isClosed()) {
                    OutboundSseEvent ping = sse.newEventBuilder()
                        .comment(s: "keep-alive")
                        .reconnectDelay(l: 15000)
                        .build();
                    session.send(ping);
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }).start();
    }
}

```



Cliente JavaScript



```
<!DOCTYPE html>
<html lang="es">
  <head><meta charset="UTF-8" /><title>Demo SSE</title></head>
  <body>
    <h2>Mensajes SSE:</h2>
    <ul id="mensajes"></ul>
    <script>
      const source = new EventSource("api/events");

      // manejador para los eventos nombrados "mensaje"
      source.addEventListener("mensaje", e => {
        const li = document.createElement("li");
        li.textContent = e.data;
        document.getElementById("mensajes").appendChild(li);
      });

      source.onerror = e => console.error("❌ Error SSE", e);
    </script>
  </body>
</html>
```



Ejemplo funcional



Ejemplo funcional

```
for (int i = 1; i <= 10; i++) {  
    session.send(sse.newEventBuilder()  
        .name(s: "mensaje")  
        .data(String.class, o: "Hola mundo " + i + "!")  
        .build());  
    Thread.sleep(millis: 1000);  
}
```



Gestión de errores



Gestión de errores

Se ejecuta cuando la conexión falla.

El parámetro `err` contiene información sobre el error ocurrido.

Se imprime en la consola del navegador un mensaje

```
const source = new EventSource("api/events");

// manejador para los eventos nombrados "mensaje"
source.addEventListener("mensaje", e => {
  const li = document.createElement("li");
  li.textContent = e.data;
  document.getElementById("mensajes").appendChild(li);
});

source.onerror = e => console.error("✖ Error SSE", e);
```



Reconexión automática



Manda un comentario invisible al cliente. No dispara ningún `addEventListener()` en JavaScript. Para que no se cierre la conexión ya que es persistente y podría cerrarse por inactividad

```
@GET
@Produces(MediaType.SERVER_SENT_EVENTS)
public void stream(@Context SseEventSink sink, @Context Sse sse) {
    new Thread(() -> {
        try (SseEventSink session = sink) {
            // Enviar 10 eventos "data"
            for (int i = 1; i <= 10; i++) {
                session.send(sse.newEventBuilder()
                    .name(s: "mensaje")
                    .data(String.class, o: "Hola mundo " + i + "!")
                    .build());
                Thread.sleep(millis: 1000);
            }
            // Mantener la conexión abierta enviando comentarios
            while (!session.isClosed()) {
                OutboundSseEvent ping = sse.newEventBuilder()
                    .comment(s: "keep-alive")
                    .reconnectDelay(i: 15000)
                    .build();
                session.send(ping);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }).start();
}
```



Ejemplo práctico

CLIENTE

```
const source = new EventSource(contextPath + "/api/events");

source.addEventListener("mensaje", function(e) {
  const mensaje = e.data;

  li.innerHTML = '<i class="bi bi-check-circle-fill me-2"></i>' + mensaje;
  li.style.display = "list-item";
  li.style.opacity = "1";

  if (timeoutId !== null) {
    clearTimeout(timeoutId);
  }

  timeoutId = setTimeout(() => {
    li.style.opacity = "0";
    setTimeout(() => {
      li.textContent = "";
      li.style.display = "none";
      timeoutId = null;
    }, 500);
  }, 4000);
});
```



Este método se activa cuando el navegador abre una conexión a /api/events para empezar a escuchar notificaciones en tiempo real (con EventSource en JavaScript).
Guarda la conexión que crea el cliente.

```
@GET @Globe @Check(Pablo Mediano Martínez)
@Produces(MediaType.SERVER_SENT_EVENTS)
public void stream(@Context SseEventSink sink, @Context Sse sse, @Context HttpServletRequest request) {
    String nombreUsuario = request.getRemoteUser(); // usuario logueado
    notificacionService.addCliente(sink, sse, nombreUsuario);
}
```

```
public String crea() { 2 usages  👤 Mario Martínez Gallego +1
    Principal p = sc.getCallerPrincipal();
    if (p != null) {
        Usuario u = usuarioDAO.buscarPorNombreUsuario(p.getName());
        libro.setUsuario(u);
        libroDAO.crea(libro);

        // Enviar notificación
        String mensaje = "El usuario " + u.getNombreUsuario() + " ha registrado el libro \"" + libro.getTitulo() + "\".";
        notificacionService.enviarMensaje(mensaje, u.getNombreUsuario());

        return "visualizaLibro?faces-redirect=true&id=" + libro.getId();
    }

    fc.addMessage(s: null, new FacesMessage(summary: "No se pudo crear el libro: usuario no autenticado"));
    return null;
}
```



```
public void enviarMensaje(String mensaje, String emisor) { 1u
    if (sse == null) return;

    OutboundSseEvent evento = sse.newEventBuilder()
        .name(s: "mensaje")
        .data(String.class, mensaje)
        .build();

    clientes.forEach(( SseEventSink sink, String usuario) -> {
        if (!sink.isClosed()) {
            if (!usuario.equals(emisor)) {
                sink.send(evento);
            }
        } else {
            clientes.remove(sink);
        }
    });
}
```

CLIENTE

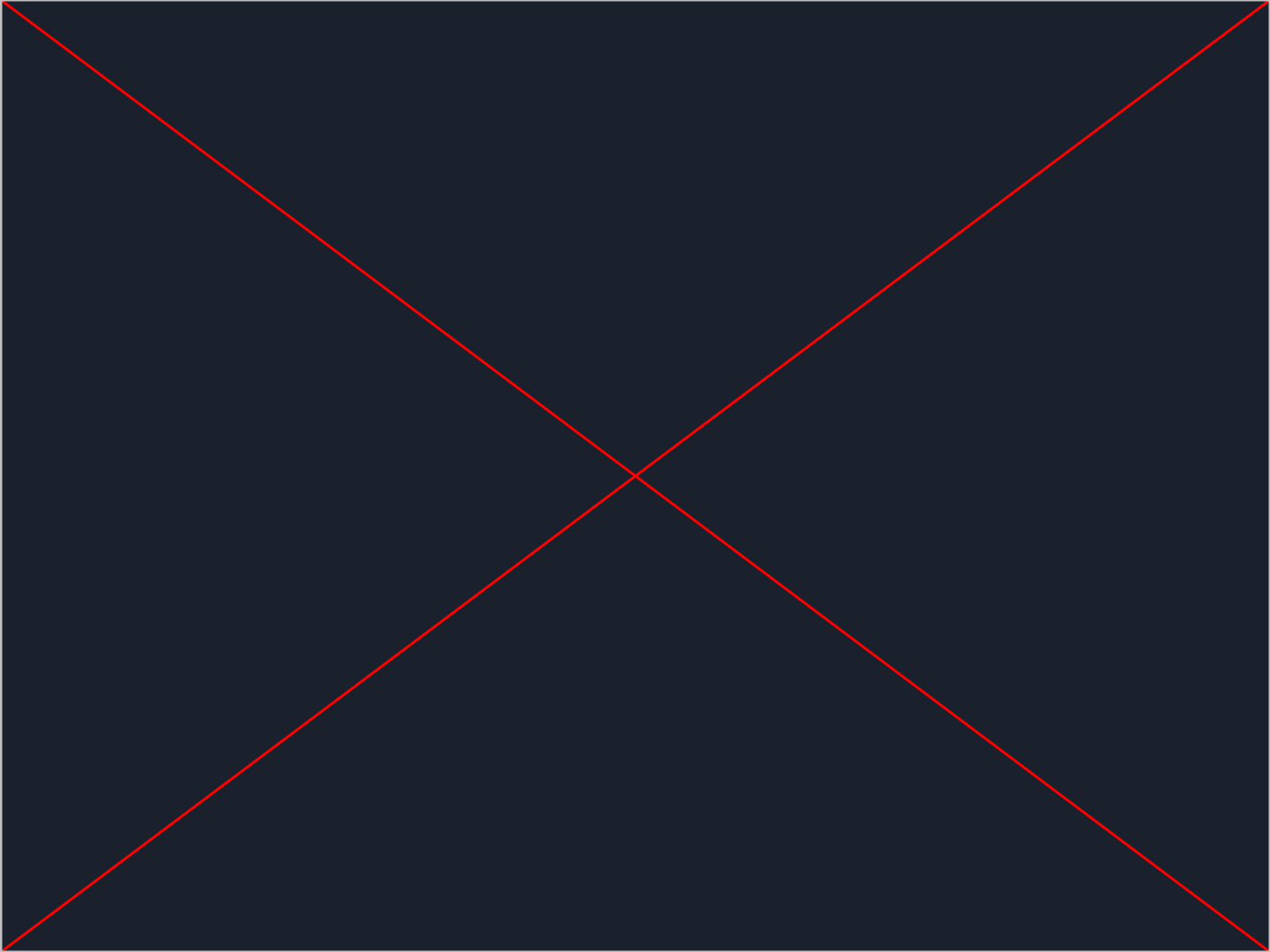
```
const source = new EventSource(contextPath + "/api/events");

source.addEventListener("mensaje", function(e) {
    const mensaje = e.data;

    li.innerHTML = '<i class="bi bi-check-circle-fill me-2"></i>' + mensaje;
    li.style.display = "list-item";
    li.style.opacity = "1";

    if (timeoutId !== null) {
        clearTimeout(timeoutId);
    }

    timeoutId = setTimeout(() => {
        li.style.opacity = "0";
        setTimeout(() => {
            li.textContent = "";
            li.style.display = "none";
            timeoutId = null;
        }, 500);
    }, 4000);
});
```





Ventajas y limitaciones de SSE

Algunas de las principales ventajas del uso de este SSE son:

- Es una solución robusta y simple en términos de conexiones unidireccionales
- El manejo de los mensajes se hace de forma automática
- No necesita bibliotecas externas

Sin embargo, su simplicidad hace que no sea la mejor opción cuando:

- Se quieren implementar conexiones bidireccionales
- Los datos enviados son complejos, ya que está optimizado a datos basados en texto
- Se quiere añadir garantías de entrega



Conclusión

SSE es una solución robusta y simple para escenarios donde el servidor necesita enviar actualizaciones en tiempo real (como notificaciones) sin esperar interacción constante del cliente. La API EventSource en el lado del cliente facilita su uso y maneja la reconexión automática. El uso conjunto con Jakarta EE 10 mejora la gestión de conexiones y la facilidad de desarrollo.

Sin embargo, su simplicidad inherente implica que SSE no es adecuado para conexiones bidireccionales ni para el manejo de datos complejos o binarios, ya que está optimizado para texto. Tampoco incluye mecanismos de garantía de entrega.



Referencias

- [Uso de eventos enviados por el servidor](#)
- [Interfaz EventSource](#)
- [Idea general de Server Sent Events](#)
- [Comparación e implementación SSE](#)
- [WebSockets vs SSE](#)
- [HTML5 SSE](#)
- [SSE Events Interfaz](#)
- [SSE JAX-RS](#)
- [Server to a Client in Jakarta EE Servlet](#)



IAG generativas utilizadas y su aporte

- Chat Gpt: Generación de código, búsqueda de fuentes
- Gemini : Investigación profunda acerca de limitaciones de SSE, búsqueda de imágenes
- Deep Seek: Afianzamiento de conceptos, comprobación de fuentes.